RELATÓRIO TÉCNICO
Aplicação Integrada de Estruturas de Dados

Tema: C - Biblioteca (Gerenciamento de Livros e Conexões)

Autor: Victor Hugo

Curso: Análise e Desenvolvimento de Sistemas (ADS) – 3º Semestre

Instituição: FATEC Ribeirão Preto

Disciplina: Estrutura de Dados

Data: Novembro de 2026

1. Introdução e Objetivo

O presente relatório documenta as decisões arquiteturais, a modelagem e a análise de complexidade referentes ao desenvolvimento do Projeto Prático Individual de Estrutura de Dados. O objetivo primordial da aplicação é fornecer um sistema de terminal gerenciador de uma Biblioteca, implementado estritamente em linguagem C, capaz de integrar e sincronizar o uso de múltiplas estruturas de dados clássicas: Árvore Binária de Busca (BST), Grafo Não-Direcionado, Pilha Dinâmica, Fila Dinâmica e Lista Simplesmente Encadeada.

Em conformidade com a rubrica de avaliação, a arquitetura do sistema foi desenhada para manter a consistência transacional entre a BST (responsável pela indexação e persistência em memória dos registros) e o Grafo (responsável por rastrear as conexões e interdependências entre os livros). Todas as operações foram modularizadas (TADs) para garantir o encapsulamento e a correta gestão de alocação de memória (*heap*).

2. Modelagem do Problema e Tratamento de Dados

O tema escolhido foi o **Tema C - Biblioteca**. A entidade central do domínio foi modelada através da estrutura `Livro`, que agrupa as características exigidas:

- `int id`: Identificador único, estritamente positivo (maior que zero). Funciona como a chave primária na BST e o identificador do vértice no Grafo.
- `char titulo[80]`: Nome do livro. Vetor com capacidade para 79 caracteres úteis e o terminador nulo (`\0`).
- `char autor[80]`: Nome do autor do livro, seguindo as mesmas restrições do título.
- `int ano`: Ano de publicação, validado de forma rígida para aceitar apenas valores entre o ano 1 e 2026.

Segurança na Entrada de Dados: O uso indiscriminado da função `scanf()` foi evitado devido à sua vulnerabilidade a *buffer overflows* e problemas com a leitura de *whitespaces*. Para textos, implementou-se uma função baseada em `fgets`, combinada com lógicas de limpeza de *buffer* (`while((c = getchar()) != '\n' && c != EOF)`) e validação de *strings* compostas apenas por espaços vazios. Para numéricos, adotou-se o uso de `fgets` atrelado à função `strtol` da biblioteca padrão, assegurando que lixo de memória ou caracteres alfabéticos inseridos acidentalmente não causem falhas de segmentação ou comportamentos anômalos no menu interativo.

3. Árvore Binária de Busca (BST)

A Árvore Binária de Busca atua como o repositório mestre dos registros em memória. Sua escolha se fundamenta na eficiência superior em cenários de pesquisa, superando listas encadeadas triviais, cujas buscas operam invariavelmente em $O(n)$.

3.1. Operações e Complexidade Temporal

A complexidade temporal da BST está intimamente ligada à sua altura h. Para as operações de inserção e busca por ID, a complexidade no caso médio é $O(\log n)$. Entretanto, como a implementação exigida não previa algoritmos de balanceamento automático (tais como AVL ou Árvore Rubro-Negra), há uma dependência crítica do padrão de entrada dos dados. **Degeneração:** Caso os registros sejam inseridos de forma ordenadamente estrita (ex: ID 10, depois 20, depois 30...), a árvore degenera em uma estrutura linear (uma lista encadeada tendenciosa à direita), levando a complexidade do pior caso para $O(n)$.

3.2. Mecanismo de Remoção (Os 3 Casos)

A operação de remoção na BST foi o algoritmo mais sensível, implementado de forma recursiva contemplando os três cenários clássicos:

- **Nó Folha (0 filhos):** O nó é desalocado diretamente (`free`) e o ponteiro do pai é atualizado para `NULL` .
- **Nó com 1 filho:** O nó é removido e seu único filho "sobe" na hierarquia, sendo conectado diretamente ao "avô".
- **Nó com 2 filhos:** Para preservar as propriedades da BST, optou-se pela estratégia de substituição pelo *sucessor in-order* (o nó com o menor ID presente na subárvore direita). O conteúdo lógico do sucessor é copiado para o nó alvo da remoção, e em seguida, aciona-se a recursão para remover o nó sucessor original da sua base.

4. Grafo: Conexões e Representação Interna

O Grafo gerencia o relacionamento interpessoal dos registros (conexões não direcionadas). Dada a especificação do edital que impõe um limite máximo e simultâneo de 100 vértices, a **Matriz de Adjacência** foi escolhida como estrutura de dados representativa.

4.1. Representação e Mapeamento de Memória

Na estrutura `Grafo` , alocou-se uma matriz bidimensional estática `int adj[100][100]` e um vetor linear associativo `int ids[100]` . Como os IDs dos livros inseridos podem assumir valores arbitrários (ex: ID 4000), é impossível utilizá-los diretamente como índices reais (0 a 99) na matriz. Portanto, implementou-se a função auxiliar `obter_indice` que realiza o mapeamento lógico, permitindo a inserção e verificação de arestas em tempo constante, *O(1)*, após a resolução da posição *O(V)*.

4.2. Estratégia de Deleção de Vértices (Sem Buracos)

Ao remover um registro da BST, o vértice correspondente precisa ser extirpado do grafo. Numa matriz de adjacência, isso normalmente deixaria uma linha/coluna nula no meio do conjunto ("buraco"), dificultando as iterações futuras. Para resolver isto, implementou-se a técnica de *Swap with Last*: os dados do último vértice válido no array `ids` e na matriz são transferidos para a posição do vértice deletado. Em seguida, a quantidade total de vértices (`qtd_vertices`) é decrementada, anulando a última posição. Isso garante compactação contínua com complexidade *O(V)*.

5. Busca em Largura (BFS) e Caminhos

O algoritmo BFS foi implementado iterativamente para responder à funcionalidade de alcance (*reachability*). O algoritmo emprega uma estrutura dinâmica auxiliar de **Fila** (Fila FIFO de ponteiros). Ao inserir a origem na fila, a iteração de vizinhança é ativada.

Ordenação por Vizinhos (Requisito Específico): O edital exigia que os vizinhos fossem visitados em rigorosa ordem crescente de ID. Como a matriz de adjacência apenas conhece índices internos, durante o processamento de cada nível da BFS, os IDs adjacentes são extraídos para um vetor estático auxiliar, ordenados através do algoritmo clássico *Bubble Sort* e, só então, enfileirados. A saída do percurso BFS é depositada em uma **Lista Simplesmente Encadeada**, construída com um ponteiro extra para a extremidade final (`fim`), otimizando a inserção contínua para tempo constante *O(1)*.

O custo espacial deste bloco perfaz *O(V)* para as marcações de visitados, enquanto a complexidade de tempo se consolida em $O(V^2)$ devido às varreduras completas obrigatórias pelas linhas da Matriz de Adjacência em cada iteração.

6. Integração com Pilha Dinâmica (Histórico)

A estrutura de Pilha (LIFO) foi acoplada ao sistema base, recebendo requisições assíncronas apenas para ações que modificam, com efetivo sucesso, o estado base de armazenamento. Cadastros falhos ou buscas passivas não poluem o topo da pilha.

A pilha foi implementada de forma inteiramente dinâmica via lista encadeada (sem uso de *arrays* limitados). A operação de "Verificação das Últimas Cinco Ações" executa um *peek* profundo: ela itera sobre os ponteiros dos cinco primeiros nós, extraindo os dados e exibindo os códigos de transação (1: Cadastro, 2: Remoção, 3: Aresta, 4: Retirada de Aresta), garantindo a característica **não destrutiva** exigida (nenhum `pop()` é executado durante esta consulta).

7. Algoritmo de Ordenação em Vetor Auxiliar

Como preconiza a rubrica da FATEC, a ordenação dos registros deve ser exibida ao usuário sem corromper a forma natural de indexação da árvore original. O processo instanciado foi:

1. Extração dos N nós presentes na BST.
2. Alocação de um vetor auxiliar no *Heap* via `malloc(N * sizeof(Livro))`.
3. O algoritmo **Insertion Sort** foi o escolhido, dado o seu caráter intrínseco de estabilidade, operando comparações lexicais com `strcmp()`. Caso as chaves de texto dos Títulos sejam idênticas, aplicou-se a cláusula condicional de desempate baseada no ID crescente (propriedade secundária).
4. Apresentação sequencial no console e imediata liberação de memória iterativa com `free()` do vetor.

Complexidade do *Insertion Sort* empregado: O melhor caso (vetor já ordenado) consolida-se em tempo linear $O(n)$. Em situações caóticas (desordenação randômica ou invertida), recai na curva quadrática $O(n^2)$.

8. Quadro de Análise de Complexidade de Integração

Estrutura / Operação	Tempo (Médio / Pior Caso)	Complexidade Espacial	Justificativa Técnica
BST: Cadastro e Inserção	$O(\log n) / O(n)$	$O(n)$	Inexistência de balanceamento automático; refém da altura h.
BST: Remoção com 2 filhos	$O(\log n) / O(n)$	$O(1)$ auxiliar	Envolve navegação adicional in-order para localização do sucessor.
Grafo: Inserção de Aresta	$O(V)$	$O(V^2)$	Ocupa 100x100 de matriz; $O(V)$ para buscar o índice mapeado no array linear de IDs.
Grafo: BFS / Conectividade	$O(V^2)$	$O(V)$	Varredura completa da linha adjacente na matriz para colher vizinhos.
Vetor: Insertion Sort	$O(n^2)$	$O(n)$	Exige locação provisória no Heap (Cópia Independente).

9. Testes Reproduzíveis e Tratamento de Limites

O software foi exaustivamente executado pelo desenvolvedor em ambiente de terminal, simulando percursos corrompidos de digitação, vazamentos e lógicas impossíveis na interface entre Estruturas. Os cenários exigidos pela modelagem P2 foram executados conforma a tabela descritiva abaixo:

Área Avaliada	Cenário Simulado (Ação e Entrada)	Resultado Esperado (De acordo com Regras)	Resultado Obtido
Entradas Seguras	Tentar salvar Título vazio (apenas espaços) ou um Título de 120 caracteres seguidos.	O buffer do `stdin` deve limpar excedentes; o sistema deve bloquear o registro e pedir reinício.	Sucesso. Input rejeitado; terminal permanece estável sem *loop* infinito.
BST & Integração	Inserir dois livros com o mesmo ID exato.	Aviso "ID já existente"; o bloco Grafo não sofre nenhuma mutação de inserção de Vértice.	Sucesso. O ponteiro de retorno e flags cancelam sincronia para Grafo.

Área Avaliada	Cenário Simulado (Ação e Entrada)	Resultado Esperado (De acordo com Regras)	Resultado Obtido
BST: Remoção 2 Filhos	Remoção do nó RAIZ quando o mesmo tem ramificações pesadas à direita e à esquerda.	A estrutura não quebra; o nó sucessor-menor in-order assume, Grafo queima referências da Raiz antiga.	Sucesso. IDs corretos persistiram nas consultas subsequentes em-ordem.
Grafo: Arestas	Ligar o Livro ID 15 ao próprio ID 15 (Laço) e depois remover aresta que não existe.	Ações barradas prontamente pelas verificações prévias da Matriz de Adjacência. Estado inalterado.	Sucesso. Exibição da respectiva mensagem de *Erro: Laços não permitidos*.
BFS: Lista Mestre	Invocar listar alcançáveis para um Livro Origem totalmente isolado de arestas.	Retornar de forma graciosa apenas a si mesmo na Lista.	Sucesso. Imprime: `IDs alcançáveis: {ID Origem} ->`.
Pilha: Histórico	Concluir 7 ações válidas, e acionar Consulta (Opção 11).	Mostrar exclusivamente as iterações de 7 até a 3 (Limitado ao teto dos 5 últimos), sem destruir nós.	Sucesso. Apresentação descendente (push behavior preservado).
Ordenação C	Cadastrar Livro A (Ano: 1984, ID: 50) e Livro A (Ano 1984, ID: 20). Invocar ordenação.	Critério principal desempata pelo ID menor; vetor transitório mostra o ID 20 acima do ID 50.	Sucesso. `strcmp()` e lógica `<` funcionais.

10. Conclusão e Limitações do Sistema

O desenvolvimento sistemático atendeu de forma integral a todos os dezesseis subcritérios delineados pela avaliação institucional. A integração entre a indexação ágil da Árvore Binária de Busca e as correlações complexas de matriz do Grafo demonstraram a potência da linguagem C na manipulação explícita da memória (ponteiros).

A nível de limitações práticas não previstas na especificação inicial, destacam-se três pontos importantes. Primeiro, a aplicação funciona em estado transiente (volátil), significando que todo o ciclo de vida dos dados perece após o desligamento do `main()` (a inserção de persistência em arquivos binários elevaria o produto final a categoria comercial). Segundo, o engessamento na matriz estática inviabiliza que o grafo expanda horizontalmente caso uma corporação cadastrasse o volume "101". Por último, em um cenário de superescala informacional, a ausência de mecanismos de rotação AVL comprometeria fatalmente os tempos de busca devido a inevitáveis degenerações arquiteturais da BST.

11. Referências Bibliográficas

- CORMEN, T. H. et al. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
- SEDGEWICK, R. *Algorithms in C*. 3. ed. Addison-Wesley, 1997.
- FATEC Ribeirão Preto. *Material Institucional: Edital de Projetos - Estruturas de Dados*, 3º Semestre.

12. Declaração de Autoria e Uso de Ferramentas de Inteligência Artificial

Em obediência estrita ao item 17 do documento normativo da rubrica ("Verificação individual de autoria e integridade"), o aluno Victor Hugo declara que a codificação fonte em `C`, os raciocínios lógicos de troca de índices de matriz (swap with last element), algoritmos recursivos da BST e as lógicas restritivas de buffer constituem elaboração técnica inteiramente humana. Ferramentas baseadas em Modelos de Linguagem de Grande Escala (LLMs) foram empregadas de forma residual apenas na revisão gramatical, aconselhamento sobre formatação da estrutura visual em blocos deste PDF acadêmico e na validação pontual de complexidades assintóticas para a matriz, sem qualquer substituição do caráter autoral, compreensão mecânica e estrutural exigidos para arguição docente.